SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	V.Rakshana	**Reg. No.:**	192425137
Date:	11.08.2026	**Duration:**	60 minutes

Code of Conduct

I V.Rakshana(192425137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Rakshana

Problem Statement 24: Smart Museum Artifact Preservation Platform

1.Abstract

The Smart Museum Artifact Preservation Platform is a web-based software system designed to support museums in digitally cataloguing, monitoring, and preserving valuable artifacts. The platform provides a centralized interface for recording artifact details, preservation conditions, conservation status, environmental observations, alerts, and maintenance history. The system is intended to help museum staff make informed preservation decisions while reducing dependence on scattered manual records. Git and GitHub are used for version control, collaborative development, feature branching, meaningful commits, merging, and remote repository synchronization. Docker is used to package the web application and its dependencies into a consistent runtime environment. Docker Compose can be used to define the application services and make local deployment repeatable.

2. Problem Statement

Museums preserve artifacts that may be affected by temperature, humidity, light exposure, handling, ageing, pests, and other environmental or operational factors. When artifact records, condition reports, inspection notes, and preservation alerts are maintained manually or across

different systems, it becomes difficult to obtain a unified view of artifact health and preservation requirements. The proposed Smart Museum Artifact Preservation Platform provides a centralized digital platform for artifact inventory, condition monitoring, preservation records, alerts, and decision support.

3. Objectives

- Develop a clear web dashboard for managing museum artifacts and preservation information.
- Maintain structured artifact records including identification, category, age, location, and condition.
- Provide artifact condition indicators and alerts for preservation risks.
- Demonstrate Git branching, meaningful commits, merging, and GitHub synchronization.
- Containerize the application using Docker for consistent execution.
- Use Docker Compose for repeatable local deployment and service configuration.
- Test the application through a browser and verify container execution.

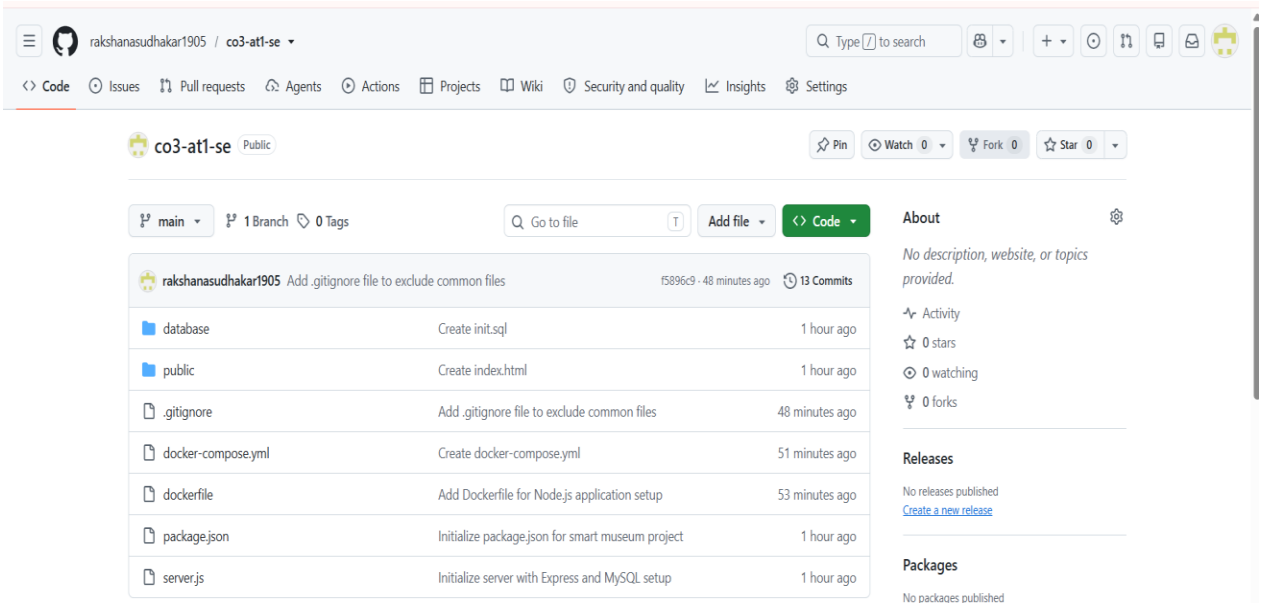

Fig 1: GITHUB Respository

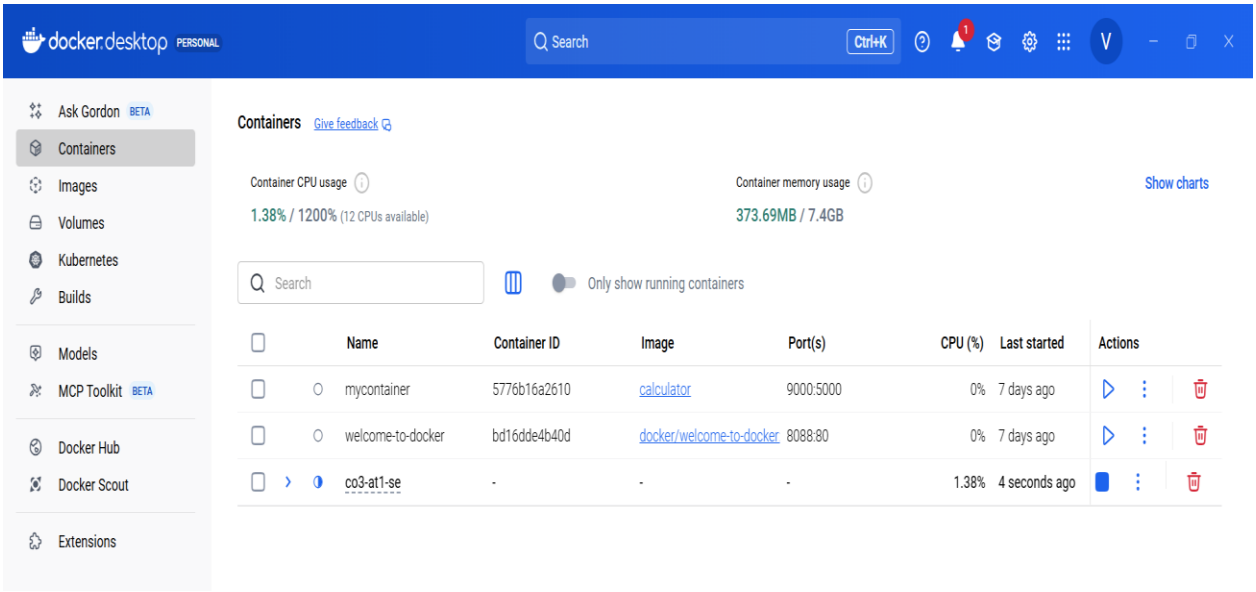

Fig 2: Docker Implementation

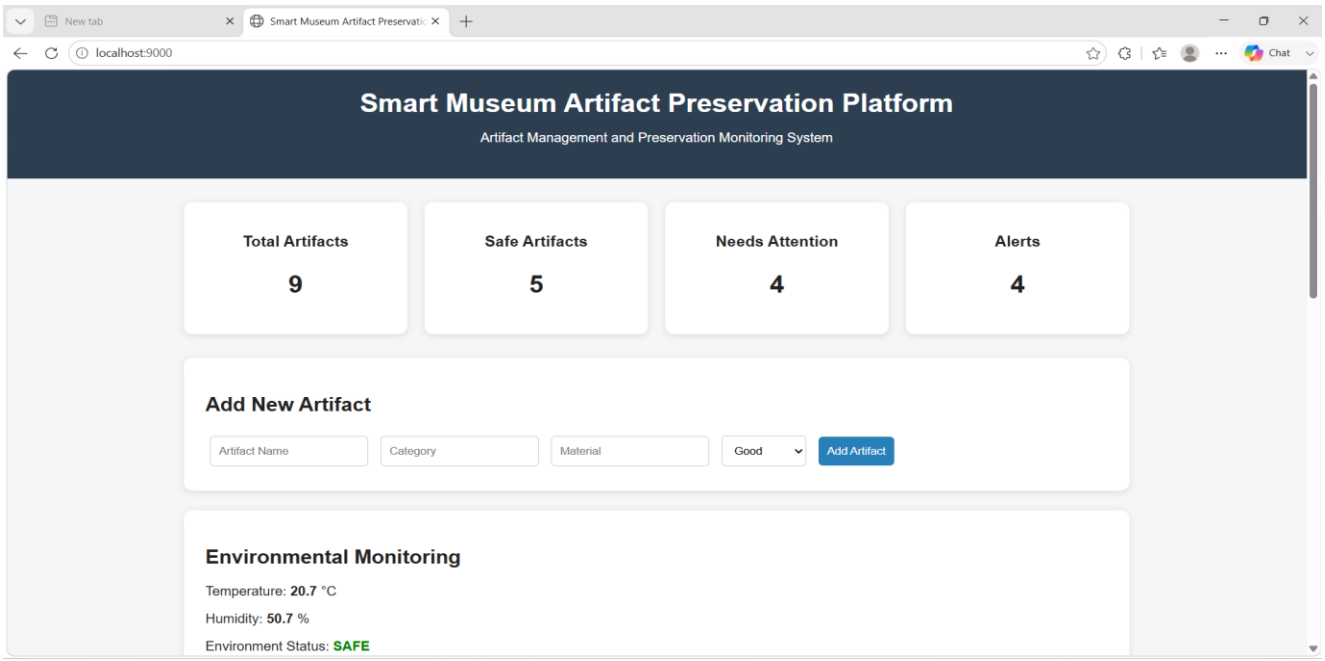

Fig 3a: Dashboard

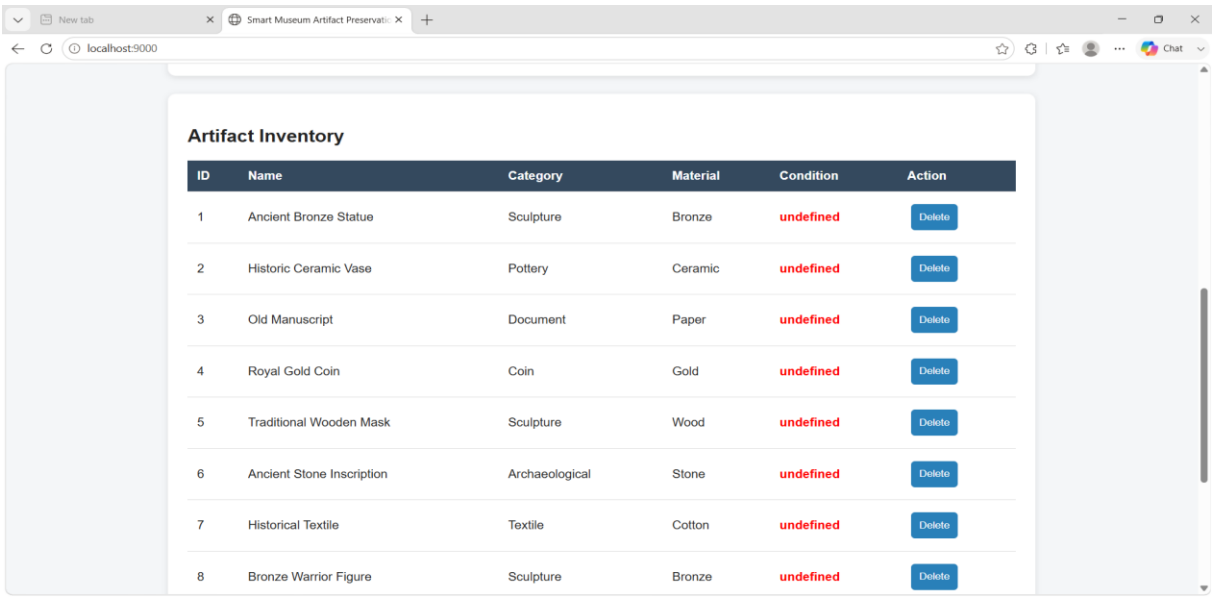

Fig 3b: Dashboard

4.Technologies Used

Technology / Tool	Purpose	Details
Python	Backend programming and application logic	3.x
HTML / CSS	Frontend interface and dashboard presentation	Web standards
JavaScript	Client-side interaction and dashboard enhancements	Browser-based
Git	Version control and local source management	Current installed version
GitHub	Remote repository and collaboration	Remote Git hosting
Docker	Application containerization	Docker Desktop
Docker Compose	Repeatable multi-service/local deployment	Compose
VS Code	Development and terminal environment	Desktop IDE

5. Application Workflow

1. The project is developed and maintained using VS Code.
2. Git tracks application source code, configuration, tests, and documentation.
3. The Flask backend receives browser requests and serves the museum dashboard.
4. Artifact records are loaded from the selected data layer.

5. Preservation rules or recorded thresholds are used to identify possible risks.
6. The dashboard displays artifact condition, environmental observations, and alerts.
7. Docker builds an image containing the application and dependencies.
8. Docker Compose starts the configured application service and maps the application port.
9. The user opens the local application in a web browser and verifies the dashboard.

6. Container Deployment Commands

- docker build -t smart-museum .
- docker run -p 5000:5000 smart-museum
- docker compose up –build
- docker ps
- docker logs smart-museum
- docker compose down

7. Deployment Workflow

1. Build the Docker image from the project directory.
2. Start the service using Docker Compose.
3. Check the running container using docker ps.
4. Open the application using the browser at http://localhost:5000/.
5. Verify dashboard pages, artifact records, preservation status, and alerts.
6. Inspect container logs if any application or startup issue occurs.
7. Stop the deployment using docker compose down after testing.

8. Conclusion

The Smart Museum Artifact Preservation Platform demonstrates the complete lifecycle of a small software project: problem analysis, system design, application development, version control, feature branching, merging, containerization, deployment, and testing. The platform provides a centralized approach to artifact records, preservation status, environmental observations, and alerts. Git and GitHub support organized development and traceable project history, while Docker and Docker Compose provide a consistent and repeatable deployment environment. The proposed architecture also provides a strong foundation for future enhancements such as database-backed records, sensor integration, role-based access, automated risk analysis, and cloud deployment.